prompt-funnel 设计书

> 一句话：在请求进入模型之前，把缺的信息显式化——该问的就问，猜的就标出来，编的一律拦掉。
>
> 版本 0.7.0 · MIT · 开源：https://github.com/sanyin/prompt-funnel

---

一、问题：大多数人不是「不会用 AI」，是「说不清自己要什么」

一个真实例子。用户说：

> 帮我写个爬虫，顺便加个代理池，再做个面板

模型不会说「你没告诉我目标网站、技术栈、要不要登录态」。它会挑一组默认值，然后给你 3000 字看起来专业、但技术栈是它替你选的实现。

这类失败不是措辞问题，是四件事缺了一两件：

缺什么	典型说法	模型会怎么做
目标	「帮我优化一下」	自己挑一个目标
交付形态	没说要代码还是文档	给一段聊胜于无的散文
约束	没说不许改接口	顺手重构了你不想动的地方
验收	没说怎样算做完	声称「已修复」，但没跑过

关键观察：模型补的这些默认值，会写进它的回答；下一轮用户把回答当前提，猜测就变成了”事实”。三步之后没人记得技术栈是谁选的。这是最常见的一种上下文污染，而且是自己造的。

二、定位：它不是提示词模板库，也不是 AI 改写器

市面上的同类做法有三类，本工具刻意都不选：

做法	问题
模板库（“高质量提示词 100 条”）	用户还得自己挑、自己填；填的时候还是不知道该写什么
AI 改写器（“让 GPT 帮你把提示词写好”）	编造不可见：它写的每一句看起来都成立，但你分不清哪句是你说的、哪句是它猜的
本工具	只做**信息显式化**：不替你写内容，只把「缺什么」变成可见、可答、可验证

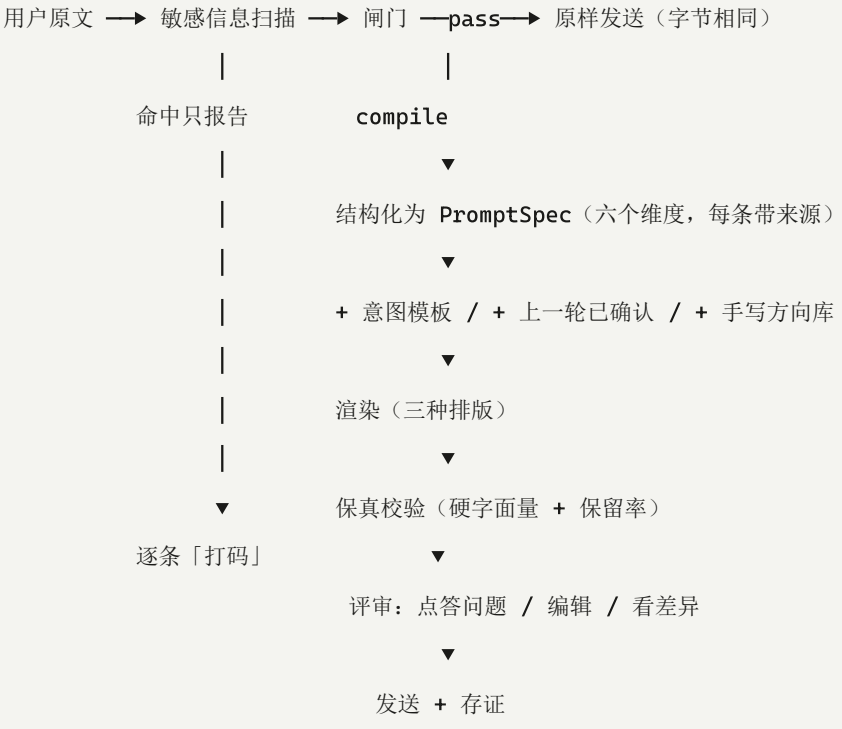
三个不可替代的机制：

1. **闸门（gate）**——先判断这句话值不值得改。清晰的原样放行（字节相同），模糊的才进入流程。**不打扰**是能不能长期用下去的前提。
2. **来源标注（provenance）**——产物是结构化对象，每个字段记着它是「用户说的」还是「规则推的」还是「默认值」。渲染时非用户的一律标「假定」。
3. **保真校验（fidelity）**——输出里出现原文没有的数字、路径、URL，直接判 error。不问模型”你有没有编”，问原文。

这三条构成了它的差异化：**不猜、可验证、不打扰。**

三、设计

3.1 流程



3.2 五个设计原则

1. **不猜**：能推导的标成「inferred」，不能推导的写成「假定」并写明若不符的后果，绝不当成事实塞进去。
2. **可验证**：要求必须编号、单条可核对；数字/路径必须能在原文找到。
3. **不打扰**：闸门先判断；清晰的请求不付出任何代价。
4. **可编辑**：用户改的是文本本身，不是跟另一个模型谈判。
5. **离线优先**：默认全部本地计算，不联网、不需要 API key。

3.3 三个具体机制（这也是最有意思的部分）

① 问题即数据，不是文案

「缺什么」不是打印一段提示语，而是一个带 `id / 问题 / 选项` 的结构。用户点选项，答案**写回对应字段**：

- 选「一份清单」→ 交付物
- 选「能跑通 / 通过测试」→ 完成标准
- 选「先做第一件」→ 约束

不经过模型，不重写用户的字。**这是”帮用户想清楚”和”替用户想”的分界线。**

② 问「什么算失败」

「怎样算做完」人人会问，「什么算失败」几乎没人问。但模型不会主动避开你没说过的坑。

所以它会问一句：`做出来不能用 / 能用但不是我想的样子 / 方向对但太粗糙 / 我也说不上来`。选前三个 → 变成一条反面验收标准（`不出现：做出来不能用`）；选最后一个 → 只撤掉问题，**不替你编标准。**

③ 把空心形容词变成选择题

「有竞争力」「有新意」无法验证——模型会给你一个**听起来**新的东西，然后就算达标了。

所以这类词会被单独拎出来：新意落在**玩法 / 视觉 / 技术**，还是「先给我三个方向挑」。选后者，提示词里会加一条「先给 3 个方向，各一句话说明取舍，等我选定再展开」——**方向由模型提，但由用户选。**

四、现状：做了什么

4.1 规模

项	数量
单元测试	128 条
离线评测用例	16 条（不需要 key，可复现）
运行时依赖	**0**
支持语言	中文 / 英文（渲染随输入语言）
版本	0.7.0

4.2 六个入口，同一个内核

入口	面向谁	说明
网页	普通人	粘一句话 → 看到缺什么 → 点几下 → 复制
浏览器扩展	普通人	ChatGPT / Claude / DeepSeek 输入框旁的按钮，答完写回输入框
CLI	开发者	`funnel compile / review / refine / continue / ideas`
MCP	各类 agent	7 个工具，宿主用自己的模型做语义工作，不需要第二个 key
Hook	Claude Code 用户	发送前附加结构，默认不拦人
Skill	Codex / DSH	让 agent 知道何时该收束

4.3 为什么是零依赖、纯本地

因为核心承诺是「可审计、不编造、不联网」。一旦把模型放进编译流程，这三条同时变弱。所以：

- 规则引擎负责**结构**（抽取、编号、标注），不负责**内容**；
- 只有在用户明确要求时，才调用用户自己的模型（可选，非默认）。

五、验证：三层，分开说

这是这份文档里最需要如实交代的部分。

5.1 第一层：可复现的（已做）

16 条手工标注用例，`npm run eval` 可复现，不需要 API key：

指标	结果
闸门与人工标注一致率	16/16
原文内容词保留率（中位）	100%
未收束字面量（编造）	0
长度开销（中位）	18 → 304 tokens

5.2 第二层：有混杂的（已做，但不该过度解读）

真实模型 A/B：deepseek-chat，60 次调用（10 用例 × 2 组 × 3 次采样）。

指标	原文直发	收束后
明确说出自己的假设	**5/30**	**18/30**
带自检 / 验收表述	2/30	17/30
输入 tokens	18	276

必须说清楚：这两个指标是关键词匹配，而收束后的提示词里本来就有「假定」「完成标准」这些词，模型可能在复读模板。所以 **5/30 → 18/30 不能读成"假设意识提升 3.6 倍"**。

但人工对照里有一段是有说服力的。同一请求「帮我写个爬虫，加代理池，做面板」：

> **原文直发**：「下面给你一套可直接运行的完整方案……」（3000 字，不说哪里是猜的）
>
> **收束后**：「**由于你未指定目标网站**，我采用 Python 技术栈，以 httpbin.org/ip 作为示例目标……」（第一句就交代了替你决定了什么）

5.3 第三层：没做的（重要）

- **没有人做人工盲评。**评分表已生成（A/B 顺序打乱、臂别隐藏），但还没有人评。
- 只测了**一个模型**、10 个用例。
- 没有测「单点提问」的对照组（这类请求根本不会进入流程，理论上应当无差别）。

所以现在的结论只到：编译是忠实的、该问的问了、开销可量化。**不主张"回答质量更好"**——那需要盲评。

六、未来：分三档

6.1 已经在做 / 三个月内

项	说明
方向库扩容	现在 26 条手写方向（游戏/工具/写作/数据/研发）。它的覆盖度就是它能帮到的人的比例
浏览器扩展实测	目前只在本地假页面验证，需要在真实站点上过一遍
模型侧盲评	把评分表评完，得到唯一一个”质量”结论
记忆	同一个人反复在同一个问题上选同一项，就不再问（需要谨慎，避免让人忘记自己答应过什么）

6.2 六到十二个月（如果验证成立）

方向	前提
「如果你其实要的是 X」三选一	现在用户还得自己描述；给候选能把”想清楚”的成本转成”选一个”
团队版（组织模板 + 审计日志）	这是唯一可能收费的点：合规场景需要”谁在什么时候发了什么”
宿主深度集成	DSH / Codex 目前只有 MCP 和 skill；如果它们开放”发送前钩子”，体验能再进一步

6.3 明确不做

- **不做 AI 改写器**（把模型塞进编译流程，回到”不可审计”）
- **不做拦截式网关**（每条消息都拦一次，用户会学会盲点确认）
- **不做”一键美化”**（没有可验证的标准）

七、落地

7.1 三类用户

用户	场景	价值
个人开发者	写需求给 AI、让 AI 改代码	少返工一轮，省下的时间远大于 250 tokens
非技术用户	在网页/扩展里把话问清楚	从”它不懂我”到”我知道该说什么”
团队	统一需求质量、留痕	可审计：每次发给模型的内容有据可查

7.2 成本

绝对开销约 **+117 ~ +272 tokens**（取决于排版），默认档最贵。按 DeepSeek 价格，1000 条/天约 ¥0.25/天——**钱基本不是问题，上下文拥挤和延迟才是。**所以提供了 `--budget`：超预算自动降级排版并明确告知。

7.3 分发

npm（`npx prompt-funnel`）、GitHub（含网页版，每次推送自动部署）、浏览器扩展（开发者模式加载，未上架）。

八、局限与风险（如实列）

1. **规则引擎不理解语义。**靠标记词判断，中文口语会漏标。漏了不会编造，会表现为保留率偏低。
2. **效果证据不足。**见 5.3：没有人工盲评、单模型、10 个用例。
3. **敏感信息靠正则。**能抓常见格式（`sk-`、`ghp_`、JWT、手机号、内网 IP），抓不到自定义格式；宁可漏报也不误报——被假警报训练出的”无脑点忽略”比漏一个更危险。
4. **浏览器扩展只在假页面验证过**，真实站点未测；失效表现是按钮无反应，不破坏页面。
5. **单人维护。**没有 CI 之外的回归测试，没有多语言 CLI 文案。
6. **一个已踩过的坑**：GitHub Pages 曾被我设成手动部署，导致”代码推了但线上没更新”。现在改为推送自动部署——**但这类”链路以为通了其实没通”的问题，在小项目里很常见**，需要在流程上防。

九、想请老师给意见的三个问题

1. **评测方法**：在没有预算做大规模人工盲评的前提下，有没有更经济的可信办法？（现在的思路是：小样本盲评 + 客观指标，不主张平均分）

2. **定位取舍**：它现在是”开发者的工具”还是”普通人的工具”？两条路的产品形态完全不同（CLI vs 网页/扩展）。要不要先定一个？
3. **安全方向**：如果往”AI 使用安全”靠，最该补的是什么？（我目前的判断是：PII 脱敏 + 注入隔离评估 + 威胁模型，而不是”提示词改写”）

附录：两分钟试用

```
npx prompt-funnel review "我想做一个像素小游戏，要有新意"
```

或直接打开网页版（无需安装）：https://sanyin.github.io/prompt-funnel/

想验证”它没有编造”，可以跑：

```
npx prompt-funnel compile "把 src/app.ts 的并发从 4 改成 8" --json
```

输出的 spec 里，每个字段都带着 `source: user | inferred | assumption`。